GOSINO

1. Project Overview

GOSINO is a top-down, rogue-like stealth-gambling game built with Python and Pygame. The player controls a gorilla character who enters a casino each day with one objective: win enough money to pay off an ever-growing debt before the casino closes — all without getting caught by the guards.

The game blends three core pillars: gambling mechanics (slot machines, dice, and roulette), stealth and evasion (avoiding patrolling AI guards), and resource management (balancing money, debt, shop items, and upgrades across multiple days). Each day the time limit shrinks and the debt grows, creating escalating pressure that forces the player to take increasingly risky bets.

2. Project Review

I got the idea from the 3D game named “clover pit” which is also a gambling roguelike game. But my game is a 2D game that added features like guards that chase the player and more minigames not just slot machines.

3. Programming Development

3.1 Game Concept

The objective of this game is to earn as much money as possible by gambling and be able to pay the debt by the end of the day without getting caught by the guards and survived as many days as possible and you can also use money to buy stats upgrades and active skill to make your stats developed as the game go.

3.2 Object-Oriented Programming Implementation

- Player class:

  Attributes: x: float, y: float, speed: float, luck: float, stealth: float, money: int

  Methods: move(dx, dy: float), update()

  Description: Represents the player character on the casino floor. It handles movements, stats, and drawing the player.
- Guard class:

  Attributes: x: float, y: float, type: str, speed: float, suspicion: float, state: str

  Methods: update(dt: float), detect(player): bool

  Description: AI enemy that patrols the casino and chases the player when suspicious. It defined the behavior of guards (each guard has a cone in front of it to detect if it sees the player or not if it does the suspicious bar will increase if the bar is filled the guard will start chasing the player and increase the speed of guards when time passes by), and drawing itself.
- Shop class:

Attributes: stock: list, inventory: dict, hot_streak_count: int

Methods: buy(item: str): bool, restock(), update()

Description: Provides an in-game overlay for purchasing upgrades and consumable items using money.

- Day System class:

  Attributes: day: int, timer: float, debt: int, closing: bool

  Methods: tick(dt: float), next_day()

  Description: Manages the per-day timer, debt escalation, and casino-closing logic.

- Message System class:

  Attributes: messages: list, queue: list

  Methods: push(msg: str), pop(): str, clear()

  Description: Two-channel notification system — floating world-space messages during exploration, and fixed UI messages visible on every screen.

- SlotMachine class:

  Attributes: bet: int, phase: str, session_profit: int

  Methods: spin(), resolve(): int, update(dt: float)

  Description: Provides an in-game overlay for slot machine mini-game with animated reels and also the winning logic for slot machines.

- Roulette class:

  Attributes: bet: int, bet_type: str, phase: str

  Methods: spin(), resolve(): int, update(dt: float)

  Description: Provides an in-game overlay for roulette wheel and interactable betting board and also the winning logic for roulette.

- DiceGame class:

  Attributes: bet: int, target: int, mode: str, final_roll: int

  Methods: roll(): int, resolve(): int, update(dt: float)

  Description: Provides an in-game overlay for dice game and the winning logic for dice game.

- CasinoMap class:

  Attributes: areas: list, walls: list, cooldowns: dict

  Methods: get_are(x, y): str, is_blocked(x, y): bool

  Description: contains the floor layout, interaction zones, player interaction in the map, and wall geometry

- ScoreSystem class:

  Attributes: score: int

  Methods: add(points: int), reset(), get(): int

  Description: get called by every gambling game on win, track cumulative gambling winnings as a score, and render the score to HUD.

- TitleScreen class:

  Attributes: fullscreen: bool, show_fps: bool

Methods: render(), handle_input()

Description: Provides a title screen that contains title, settings, and animated background screen.

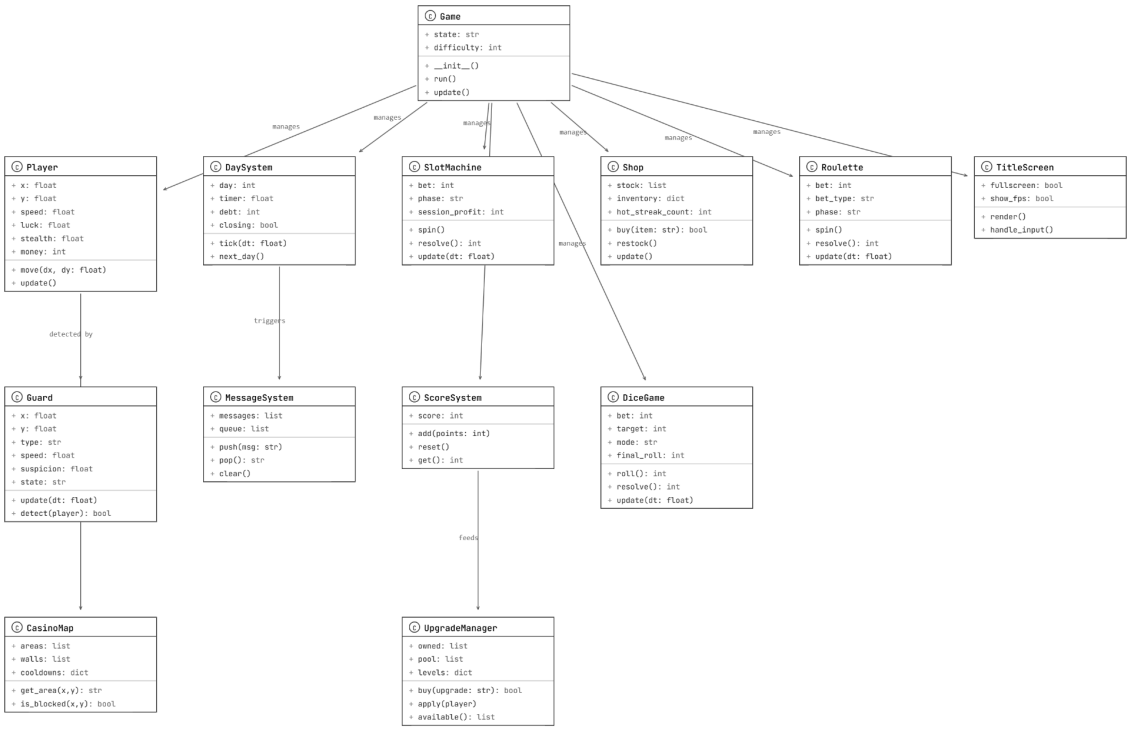
- UpgradeManager:

  Attributes: owned: list, pool: list, levels: dict

  Methods: buy(upgrade: str): bool, apply(player), available(): list

  Description: Handling the upgrades screen after each day ends.

UML class Diagram



3.3 Algorithms Involved

1. Guard Vision and Suspicion Accumulation

Each frame, every guard checks how far and what direction the player is. If the player is close enough and in front of the guard, the guard becomes more suspicious. Suspicion increases faster when the player is directly in front, and slower when the player is slightly to the side. If the player is not seen, suspicion slowly decreases over time. Once suspicion gets high enough, the guard starts chasing the player and alerts nearby guards.

2. Guard State Machine (Patrol / Alert / Search / Chase)

Guards use four behavior states: patrol, alert, search, and chase. On patrol, they walk around and randomly change direction. In alert mode, they move more carefully for a short time. In search mode, they go to the last place they saw the player and

look around. In chase mode, they run directly after the player but will slow down over time. In all states, they can slide along walls when moving so they don't get stuck.

3. **Gambling Outcome Pre-computation**

All three gambling games decide the result before the animation starts. The slot machine randomly picks whether the player wins or loses and sets the symbols. The dice game generates a number and adjusts it if the player has special items. The roulette also decides the final number and color right away. This ensures that any shop items or bonuses always work correctly, no matter what happens during the animation.

4. **Difficulty Scaling**

The game gets harder over time. While the player is exploring, a timer increases continuously. Every set amount of time, the difficulty level goes up, which makes guards move faster. At the same time, each new day becomes harder by increasing the debt they need to pay. This creates a steady increase in difficulty both during a day and across multiple days.

5. **Player Wall Collision Sliding**

Both the player and guards handle collisions by checking movement one direction at a time. They first try moving horizontally and cancel it if it hits a wall, then do the same for vertical movement. This lets them slide smoothly along walls instead of getting stuck.

4. Statistical Data (Prop Stats)

4.1 Data Features

The following five gameplay features will be tracked during each session. All features are measured per-event and accumulate to well over 100 rows across a typical play session.

Feature	Why is this data good? What can it be used for?	How will you obtain 100+ values of this feature data?	Which variable (and which class) will you collect this from?	How will you display this feature data?
Gambling Event Outcome	Reveals which mini-game is most profitable, what bet sizes lead to winning, and how shop items affect outcome rates. Useful for balancing the game's economy.	Every spin/roll/deal fires a log call inside _resolve(). A 10-minute session generates 50–150 gambling events across slots, dice, and roulette easily exceeding 100 rows.	outcome, bet_amount, payout, game_type — collected from SlotMachine, DiceGame, Roulette via their _resolve() methods.	Bar chart: win rate % per game type. Scatter plot: bet size vs payout amount.

Feature	Why is this data good? What can it be used for?	How will you obtain 100+ values of this feature data?	Which variable (and which class) will you collect this from?	How will you display this feature data?
Guard Detection Event	Shows which map areas are most dangerous and how often guards spot the player. Useful for identifying high-risk zones and balancing guard placement.	Logged each time suspicion crosses a threshold in Guard.see_player(). Multiple guards x multiple days = 100+ rows quickly in a typical session.	suspicion_level, player_x, player_y, guard_type — collected from the Guard class inside see_player().	Scatter plot of player (x, y) at detection moment overlaid on casino map — a spatial heatmap of danger zones.
Player Money Snapshot	Provides a time-series of player wealth across each day. Shows crash points, recovery patterns, and how debt pressure changes behaviour over time.	A timer in the main loop records balance every 10 real-time seconds. A 15-minute session = 90+ snapshots — easily 100 rows across two sessions.	player_money, net_balance — collected from Player.money and DaySystem.debt in the main game loop.	Line graph: player balance (y-axis) over time in seconds (x-axis), one line per day.
Shop Purchase Event	Shows which items players buy most and when. Reveals which items are considered valuable and helps balance item pricing and availability.	Logged inside Shop._buy() on every successful purchase. With 4 items per day and multiple days per session, 100 rows accumulates across 3–4 sessions.	item_name, item_type, item_cost, day — collected from the Shop class inside _buy().	Bar chart: purchase frequency per item, grouped by day number.
Day Completion Summary	Provides a per-run snapshot of overall performance. Useful for comparing sessions and identifying which days players most often fail.	One row written per day at day-end. At 7 days per run x multiple sessions, 100 rows is reached after approximately 15 complete sessions.	day_number, ending_money, debt_paid, total_wins, total_losses — collected from DaySystem and aggregated in Game._handle_exit().	Summary statistics table: mean, min, max, std for money earned and bets placed per day.

Feature Name	Mean	Median	Std. Deviation	Min	Max
Gambling Event Outcome	Average payout per gambling event across all game types	Middle payout value, shows typical earnings without outlier jackpots	Spread of payout amounts, high value means volatile, unpredictable	Lowest payout recorded	Highest single payout, reveals jackpot frequency
Guard Detection Event	Average suspicion level at the moment of detection	Typical suspicion threshold that triggers a chase	How consistently guards detect the player	Lowest suspicion level that still triggered an alert	Highest suspicion recorded before chase began
Player Money Snapshot	Average player balance across the whole session	Typical balance less affected by brief spikes from big wins	How wildly the player's money fluctuates, high means risky play style	Lowest balance recorded , shows how close the player got to bankruptcy	Peak balance reached during the session
Shop Purchase Event	Average cost of items purchased per session	Typical item price the player chooses to spend	Variation in spending, shows whether players buy cheap or expensive items consistently	Cheapest item purchased	Most expensive single purchase made
Day Completi on Summary	Average days survived per run	Typical run length, less affected by unusually long or short runs	Consistency of run length, low means most runs end around the same day	Shortest run (earliest day the player failed)	Longest run survived across all sessions

	Feature Name	Graph Objective	Graph Type	X-axis	Y-axis
Graph 1	Gambling Event Outcome	Show the distribution of payout amounts across all gambling events — reveals whether wins are clustered at small payouts or spread across amounts.	**Histogram (Distribution)**	Payout amount ($)	Frequency (number of events)
Graph 2	Guard Detection Event	Show which areas of the casino floor are most dangerous by mapping where the player was when detected.	**Scatter Plot (Relation)**	Player X position (pixels)	Player Y position (pixels)
Graph 3	Shop Purchase Event	Compare how often each shop item is purchased to reveal player preferences and item value perception.	**Bar Graph (Proportion)**	Item name	Purchase count
Graph 4	Player Money Snapshot	Show how player balance changes over time within a single day across multiple sessions.	**Line Graph (Time-series)**	Time elapsed (seconds)	Player money balance ($)

4.2 Data Recording Method

The game will save important data into CSV files using Python's built-in csv module. A StatsLogger class will handle this by writing data to several CSV files in a /stats/ folder. Each time something important happens, a new row with a timestamp is added so data from different game sessions can be tracked. Logging happens at key moments in the game, such as when a gambling result is decided, when guards become suspicious, when money is updated over time, when the player buys items, and when a new day begins.

4.3 Data Analysis Report

After collecting data from at least 10 play sessions, the data will be analyzed using basic graphs and charts. Bar charts will compare win rates between the different games (slots, dice, and roulette) to see which one is most profitable. Scatter plots

will show the relationship between bet size and payouts. Line graphs will track the player's money over time to see common patterns. A heat map will show where guards detect the player most often on the map. Another bar chart will show how often each shop item is bought at different stages of the game.

5. Project Planning Timeline

Project Week	Task
Week 1	Proposal submission / Project initiation
Week 2	Core architecture: Player, Guard, CasinoMap, DaySystem, and main game loop running
Week 3	Gambling mini-games: implement SlotMachine, DiceGame, and Roulette with full input handling, animation, and payout logic
Week 4	Shop and upgrade systems Shop item pool, consumable inventory, UpgradeManager post-day screen
Week 5	Guard AI polish, suspicion system, state machine transitions, vision cone rendering, difficulty scaling
Week 6	Statistics pipeline StatsLogger class, CSV logging at all event points, preliminary data collection from test sessions
Week 7	Submission week (Draft)

6. Document Version

Version	**Date**	**Editor**	**Notes**
1.0	13 March 2025	Jedsada Ungsunantwiwat	Initial proposal submission
2.0	2 April 2025	Jedsada Ungsunantwiwat	Adding Statistical values Table, Add timeline, Add attributes and methods to 3.2

Date	Name	Feedback
30/03	Peraya	- **Missing Statistical Values Table:** You identified the features and graphs, but you are

		missing the mandatory **Table of Statistical Values** (showing Mean, Median, Std. Deviation, Min, and Max) for your features. - **Incomplete Timeline:** Your Section 5 is empty. A project proposal must have a week by week breakdown of development tasks. - **Section 3.2:** While you listed many classes, you did not provide the specific **Attributes** and **Methods** for each.